

Merge Sort Questões

Disciplina: Estrutura de Dados III

Trabalho de Revisão

Alunos:

- Matheus Witte Ditz
- Gabriel Rizzatto
- Andrey Dalla Costa

1. ENADE 2019

O MergeSort é um método de ordenação que combina dois vetores ordenados e cria um terceiro vetor maior também ordenado. O algoritmo abaixo apresenta essa ideia e combina os vetores $a[lo..mid]$ e $a[mid+1..hi]$ no vetor $a[lo..hi]$.

```
public class MergeSort {  
  
    private static Comparable[] aux;  
  
    public static void merge(Comparable[] a, int lo, int mid, int hi) {  
  
        int i = lo, j = mid+1;  
  
        for (int k = lo; k <= hi; k++)  
            aux[k] = a[k];  
  
        for (int k = lo; k <= hi; k++) {  
  
            if (i > mid)  
                a[k] = aux[j++];  
  
            else if (j > hi )  
                a[k] = aux[i++];  
  
            else if (aux[j].compareTo(aux[i]) < 0)  
                a[k] = aux[j++];  
  
            else  
                a[k] = aux[i++];  
  
        }  
    }  
}
```

```
}

public static void sort(Comparable[] a) {

    aux = new Comparable[a.length];

    sort(a, 0, a.length - 1);

}

private static void sort(Comparable[] a, int lo, int hi) {

    //implementação

}

}
```

Considerando o código apresentado, a implementação do protótipo do método sort da classe MergeSort é

a)

```
if (hi == lo)

    return;

int mid = lo + (hi - lo)/2;

sort(a, lo, mid);

sort(a, mid, hi);

merge(a, lo, mid, hi);
```

b)

```
if (hi > lo)

    return;

int mid = lo + (hi - lo)/2;

sort(a, lo, mid);

sort(a, mid, hi);

merge(a, lo, mid, hi);
```

c)

```
if (hi <= lo)

    return;

int mid = lo + (hi - lo)/2;

sort(a, lo, mid);

sort(a, mid, hi);

merge(a, lo, mid, hi);
```

d)

```
if (hi > lo)

    return;

int mid = lo + (hi - lo)/2;

sort(a, lo, mid);

sort(a, mid+1, hi);

merge(a, lo, mid, hi);
```

e)

```
if (hi <= lo)

    return;

int mid = lo + (hi - lo)/2;

sort(a, lo, mid);

sort(a, mid+1, hi);

merge(a, lo, mid, hi);
```

Passo 1 — Caso base

Toda função recursiva precisa de uma condição de parada para evitar chamadas infinitas. No Merge Sort, um subvetor que possui apenas um elemento já está ordenado e não precisa mais ser dividido.

Isso acontece quando `hi` é menor ou igual a `lo`:

```
if (hi <= lo)
    return;
```

Quando essa condição é verdadeira, a função encerra aquela chamada recursiva.

Passo 2 — Divisão do vetor

Depois de verificar o caso base, é necessário encontrar o ponto central do intervalo. Para isso, utiliza-se:

```
int mid = lo + (hi - lo) / 2;
```

Essa expressão produz o mesmo resultado que $(lo + hi) / 2$, mas possui a vantagem de evitar um possível **overflow de inteiros** no cálculo de $lo + hi$ quando os índices são muito grandes.

O vetor é então dividido em duas partes:

```
a[lo..mid]
a[mid+1..hi]
```

Passo 3 — Ordenação recursiva

Cada uma das duas partes deve ser ordenada separadamente por meio de novas chamadas ao método `sort`:

```
sort(a, lo, mid);
sort(a, mid+1, hi);
```

É importante observar que a segunda chamada começa em `mid + 1`. Isso ocorre porque o método `merge` do enunciado trabalha explicitamente com os intervalos `a[lo..mid]` e `a[mid+1..hi]`.

Passo 4 — Intercalação

Depois que as duas metades estiverem ordenadas, elas precisam ser combinadas novamente. Essa etapa é realizada pelo método:

```
merge(a, lo, mid, hi);
```

Portanto, a implementação correta é:

```
if (hi <= lo)
    return;

int mid = lo + (hi - lo) / 2;

sort(a, lo, mid);
sort(a, mid+1, hi);

merge(a, lo, mid, hi);
```

Resposta correta: **Alternativa E**

2. Questão 33 — Poscomp 2013

Considere o algoritmo a seguir.

MERGESORT(V, i, j)

- (1) Se $(i < j)$ então
- (2) $m = (i+j)/2$;
- (3) MERGESORT(v, i, m);
- (4) MERGESORT(v, m+1, j);
- (5) MESCLAR(v, i, m, j);
- (6) Fim;

Sobre o comportamento assintótico do algoritmo de ordenação Merge Sort, assinale a alternativa que apresenta, corretamente, sua complexidade.

- (A) $O(\log n)$
- (B) $O(n \log n)$
- (C) $O(n^2)$
- (D) $O(n^3)$
- (E) $O(2^n)$

Para determinar o comportamento assintótico desse algoritmo, podemos utilizar a análise da árvore de recursão ou o **Teorema Mestre**.

Divisão do problema

O algoritmo sempre divide o intervalo aproximadamente pela metade:

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$$

Esse processo continua até que os subvetores possuam apenas um elemento. Como o tamanho do problema é reduzido pela metade a cada etapa, a árvore de recursão possui aproximadamente:

$$O(\log n)$$

níveis.

Custo da intercalação

Em cada nível da árvore de recursão, o algoritmo realiza a operação de **MESCLAR**.

Durante a intercalação, os elementos precisam ser percorridos para combinar as duas partes ordenadas. Considerando todos os subvetores de um mesmo nível, o trabalho total realizado é:

$$O(n)$$

Portanto, temos aproximadamente $O(\log n)$ níveis, cada um com custo $O(n)$:

$$O(n) \times O(\log n) = O(n \log n)$$

Assim, a complexidade assintótica do Merge Sort é:

$$O(n \log n)$$

Resposta correta: **Alternativa B — $O(n \log n)$**

3. Questão 15 — Poscomp 2015

Considere o algoritmo que implementa o seguinte processo: uma coleção desordenada de elementos é dividida em duas metades e cada metade é utilizada como argumento para a reaplicação recursiva do procedimento. Os resultados das duas reaplicações são, então, combinados pela intercalação dos elementos de ambas, resultando em uma coleção ordenada. Qual é a complexidade desse algoritmo?

- (A) $O(n^2)$
- (B) $O(n^{(2n)})$
- (C) $O(2^n)$
- (D) $O(\log n \times \log n)$

(E) $O(n \times \log n)$

Essa descrição corresponde ao funcionamento do **Merge Sort**, também conhecido como ordenação por intercalação.

Reconhecimento do algoritmo

As principais características apresentadas no enunciado são:

- divisão da coleção em duas metades;
- aplicação recursiva do procedimento em cada metade;
- combinação das duas partes por meio de intercalação;
- obtenção de uma coleção ordenada ao final do processo.

Essas são justamente as etapas fundamentais do Merge Sort.

Complexidade

Como o problema é dividido pela metade a cada etapa, são necessários aproximadamente $O(\log n)$ níveis de divisão.

Em cada nível, os elementos precisam ser percorridos durante o processo de intercalação, resultando em um custo de $O(n)$.

Assim:

$$O(n) \times O(\log n) = O(n \log n)$$

Portanto, a complexidade do algoritmo é:

$$O(n \log n)$$

Resposta correta: **Alternativa E — $O(n \times \log n)$**

4. Qual é a complexidade de tempo do merge sort nos casos melhor, médio e pior? Explique por que ela não muda entre os casos.

A complexidade de tempo do Merge Sort é **$O(n \log n)$** no melhor caso, no caso médio e no pior caso.

Caso	Complexidade
Melhor caso	$O(n \log n)$
Caso médio	$O(n \log n)$
Pior caso	$O(n \log n)$

Por que a complexidade não muda?

A principal razão está na própria estrutura do Merge Sort.

Diferentemente de alguns algoritmos de ordenação, como o Insertion Sort, que pode apresentar comportamento $O(n)$ quando o vetor já está ordenado, o Merge Sort tradicional continua realizando suas etapas de divisão e intercalação independentemente da disposição inicial dos elementos.

O algoritmo sempre começa dividindo o vetor em duas partes. Essas partes são novamente divididas até que se obtenham subvetores com apenas um elemento.

Depois disso, as partes são gradualmente intercaladas para formar vetores maiores e ordenados.

Portanto, mesmo que o vetor original já esteja completamente ordenado, o Merge Sort ainda realizará a divisão recursiva:

```
n
↓
n/2
↓
n/4
↓
...
↓
1
```

e depois realizará as etapas de intercalação.

A quantidade de níveis da recursão é aproximadamente $O(\log n)$ e o trabalho realizado em cada nível é $O(n)$.

Assim:

$$O(\log n) \times O(n) = O(n \log n)$$

A disposição inicial dos elementos pode alterar o resultado das comparações realizadas durante algumas intercalações, mas não elimina a estrutura principal do algoritmo.

Conclusão

O Merge Sort apresenta complexidade **$O(n \log n)$ nos três casos** porque sua estratégia de divisão e intercalação é aplicada independentemente de o vetor estar ordenado, parcialmente ordenado ou completamente desordenado.

5. Propriedades do Merge Sort Aplicado a Vetores

Sobre o merge sort aplicado a vetores, é correto afirmar que:

- a) É in-place e instável.
- b) É in-place e estável.

- c) Usa $O(n)$ de memória auxiliar e pode ser implementado de forma estável.
- d) Usa $O(\log n)$ de memória auxiliar e é sempre instável.
- e) Não usa memória auxiliar, pois ordena por trocas.

Consumo de memória

Um algoritmo *in-place* é aquele que utiliza apenas uma quantidade adicional de memória considerada constante, geralmente representada por **$O(1)$** , além do espaço ocupado pelo próprio vetor.

Na implementação tradicional do Merge Sort para vetores, é necessário utilizar um vetor auxiliar durante a etapa de intercalação.

No código apresentado na primeira questão, isso pode ser observado na declaração:

```
private static Comparable[] aux;
```

Esse vetor auxiliar possui tamanho proporcional ao vetor original. Dessa forma, o Merge Sort tradicional utiliza:

$O(n)$

de memória auxiliar.

Por esse motivo, essa implementação não é considerada *in-place*.

Estabilidade

Um algoritmo de ordenação é considerado **estável** quando elementos que possuem a mesma chave mantêm sua ordem relativa original após a ordenação.

O Merge Sort pode ser implementado de forma estável.

Durante o processo de intercalação, quando dois elementos possuem chaves iguais, basta escolher primeiro o elemento que pertence à metade esquerda.

Por exemplo, supondo que dois elementos de mesma chave sejam:

A_1 A_2

e A_1 apareça originalmente antes de A_2 , uma intercalação estável deve preservar essa ordem:

A_1 A_2

em vez de trocar suas posições.

Conclusão

Na implementação tradicional do Merge Sort utilizando um vetor auxiliar:

- a memória auxiliar é **$O(n)$** ;
- o algoritmo pode ser **estável**;
- o algoritmo não é *in-place*.

Resposta: **C**

6. [1- POSCOMP 2015 – CENTRO DE SELEÇÃO – UFG]

Quais destes algoritmos de ordenação têm a classe de complexidade assintótica, no pior caso, em $O(n \log n)$?

- (A) QuickSort, MergeSort, e HeapSort
- (B) QuickSort e SelectionSort
- (C) MergeSort e HeapSort
- (D) QuickSort e BubbleSort
- (E) QuickSort, MergeSort e SelectionSort

Análise dos algoritmos

Merge Sort

O Merge Sort divide o vetor aproximadamente pela metade e realiza a intercalação das partes ordenadas.

Sua complexidade no pior caso é **$O(n \log n)$** .

Heap Sort

O Heap Sort utiliza uma estrutura de dados chamada **Heap Binário**.

Durante a ordenação, as operações de ajuste e remoção no heap possuem custo relacionado à altura da estrutura, que é **$O(\log n)$** . Essas operações são realizadas para os elementos do vetor.

Assim, sua complexidade no pior caso é:

$O(n \log n)$

Quick Sort

O Quick Sort possui desempenho que depende da forma como o pivô divide o vetor. No pior caso, as partições podem ficar extremamente desbalanceadas. Isso pode acontecer, por exemplo, quando o pivô escolhido é repetidamente o menor ou o maior elemento.

Nesse caso, a complexidade chega a:

$O(n^2)$

Selection Sort

O Selection Sort procura o menor elemento e o coloca na posição correta, repetindo esse processo para as posições seguintes.

Esse processo exige aproximadamente:

$O(n^2)$

comparações no pior caso.

Bubble Sort

O Bubble Sort compara elementos adjacentes e realiza várias passagens pelo vetor.

No pior caso, sua complexidade é:

$O(n^2)$

Portanto, os algoritmos que possuem complexidade **$O(n \log n)$ no pior caso** são o **Merge Sort e o Heap Sort**.

Resposta correta: **Alternativa C — MergeSort e HeapSort**